

Amira Atiah S24109222

CS 3081 · ARTIFICIAL INTELLIGENCE · FALL 2026

Assignment 1 — Search Strategies

Version B · BFS · DFS · Greedy Best-First · A*

Dr Naila Marir

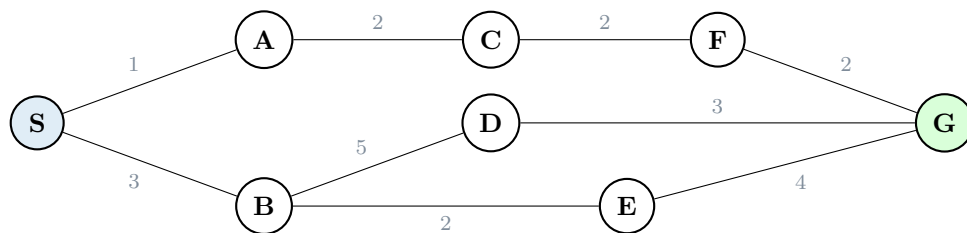
Instructor: Dr Naila Marir**Total marks:** 100**Section:** 3081-2 (CRN 10481)

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. S, A, C, G .

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is S and the goal state is G . Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

(a) Breadth-first search

[8 marks]

List the order in which states are expanded by BFS, and give the path it returns.

Expansion: S, A, B, C, D, E, F, G
 Answer: Path: S, B, D, G

(b) Depth-first search

[8 marks]

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

Expansion: S, A, C, F, G

Answer:

Path: S, A, C, F, G

(c) Comparison

[6 marks]

BFS and DFS returned different paths. State which path is shorter in number of steps, and explain why BFS is guaranteed to find that path while DFS is not.

Answer:

BFS's path has 3 steps and DFS's has 4, so BFS is shorter. BFS expands every node at depth d before any node at depth $d+1$. The first time it reaches G is therefore along a path with the fewest edges. DFS commits to one branch as deep as it can go and returns the first path it stumbles on, with no guarantee about its length.

(d) Frontier data structures

[3 marks]

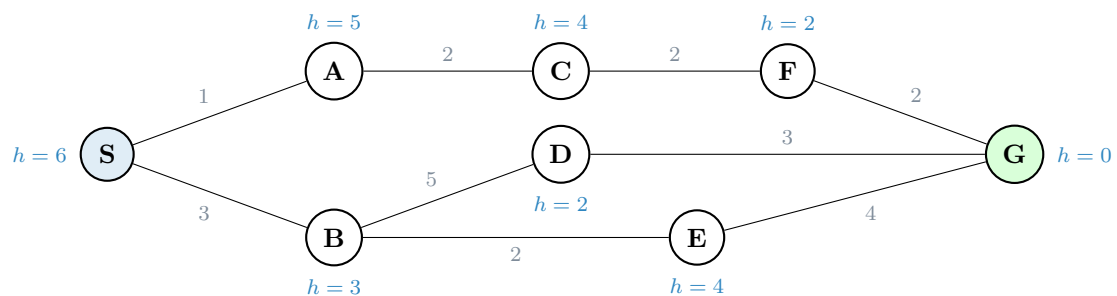
BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behaviour you observed.

Answer:

BFS uses a FIFO queue and DFS uses a LIFO stack. The queue makes the shallowest node expand next, so BFS sweeps level by level. The stack makes the deepest node expand next, so DFS dives down one branch before backtracking.

2 Informed search: Greedy Best-First and A*

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G .



(a) Greedy best-first search

[8 marks]

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

Expansion: S, B, D, G

Answer:

Path: S, B, D, G

(b) A* search

[12 marks]

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

Order	Node	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
1	S	0	6	6
2	A	1	5	6
3	B	3	3	6
4	C	3	4	7
5	F	5	2	7
6	G	7	0	7

Order of expansion: S, A, B, C, F, G

Path returned: S, A, C, F, G

Total cost: 7

(c) Why they differ

[6 marks]

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Answer:

Greedy looks only at $h(n)$ and ignores $g(n)$. B and D look close to the goal, so it rushes through the expensive B–D edge (5) and returns cost 11. A* adds g to h , so the heavy cost of going through B and D shows up in f and it finds the cost-7 path. Greedy expanded fewer nodes (4 vs 6) but returned a worse path. A* spent more expansions to get the optimal one.

3 Admissibility and optimality**(a) Definition**

[5 marks]

Define an **admissible** heuristic. Your definition must be precise enough that someone could check a given heuristic against it.

Answer:

A heuristic h is admissible if, for every node n , $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost of the cheapest path from n to a goal. In particular $h(\text{goal}) = 0$. It never overestimates the remaining cost.

(b) Testing the heuristic

[10 marks]

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G , and decide whether the given h is admissible.

Node	S	A	B	C	D	E	F	G
$h(n)$ given	6	5	3	4	2	4	2	0
$h^*(n)$ true	<u>7</u>	<u>6</u>	<u>6</u>	<u>4</u>	<u>3</u>	<u>4</u>	<u>2</u>	<u>0</u>
$h \leq h^*$?	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>	<u>yes</u>

Is h admissible? h is admissible.

If not, name every node that violates the condition: No node violates the condition.

(c) Consequence

[6 marks]

Suppose a heuristic *overestimates* the true cost at exactly one node. Explain what this can do to the solution A* returns, and why uniform-cost search (A* with no heuristic) could never suffer from this problem.

Answer:

If the overestimated node lies on the optimal path, its f is inflated and A^* may delay expanding it. Another node can then reach the goal first, and A^* returns a suboptimal solution. UCS is A^* with $h = 0$ everywhere. Since $0 \leq h^*(n)$ always holds, it can never overestimate, so it is always admissible. It expands nodes in order of g , so the first time the goal is removed from the frontier, its cost is the cheapest.

(d) A perfect heuristic

[3 marks]

Suppose $h(n) = h^*(n)$ for every node. How does A^* behave in this case? Justify your answer in one sentence.

Answer:

With $h = h^*$, $f(n) = g(n) + h^*(n)$ is the cost of the cheapest solution through n . Every node on an optimal path has f equal to the optimal cost C^* and every other node has $f \geq C^*$. A^* therefore heads straight down an optimal path with essentially no wasted expansions (apart from ties).

4 Comparing the four strategies**(a) Properties table**

[8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *complete* and *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Complete?	Optimal?
Breadth-first	FIFO queue	Yes, if the branching factor is finite	Yes, only if all step costs are equal
Depth-first	LIFO stack	No in infinite, yes in finite with visited set	No
Greedy best-first	Priority queue ordered by h	Yes in a finite space with graph search; no in infinite spaces or tree search	No
A^*	Priority queue ordered by $f = g + h$	Yes, if branching is finite	Yes, if h is admissible

(b) Choosing a strategy

[13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

- A delivery drone must find the cheapest flight path between two warehouses, and the straight-line distance to the destination is known for every waypoint.
- A web crawler must find *any* page containing a keyword. The link graph is very deep, memory is tight, and the shortest link path is not important.
- A game AI must reach a target square quickly on a large map. A good distance estimate exists, but the move does not need to be the cheapest — only fast to compute.

Answer:

i. A^* . The drone needs the cheapest path, which requires an optimal algorithm. Straight-line distance never overestimates real path cost, so it is admissible, and A^* is optimal with an admissible heuristic.

ii. DFS. The graph is deep, memory is tight and the shortest path doesn't matter. DFS stores only one path plus siblings (memory linear in depth), whereas BFS would store a huge frontier. Its lack of optimality doesn't matter here. Use a visited set to avoid cycles.

iii. Greedy best-first. It only needs to be fast, and a good estimate exists. Greedy follows the heuristic straight at the goal and expands few nodes. It gives up optimality, which isn't required.

(c) Short answer

[4 marks]

BFS is complete and optimal for uniform step costs, but is often impractical on large problems. State the main reason, and name one property of the search problem that makes it worse.

Answer:

BFS keeps every node of the current level in memory, so both its memory and time grow exponentially. A larger branching factor b or a greater solution depth d makes it worse.

Submission. Submit a single PDF to Blackboard by the due date. Handwritten work is acceptable if scanned clearly. Include your name and student ID on the first page.

Q1: 25 Q2: 26 Q3: 24 Q4: 25 **Total: 100 marks**